



6.1. Data Normalization

Previous Section:

 [5.2. Data Grouping and Transforming](#)

Contents:

[1. Why Do We Need Normalization?](#)

[2. Common Types of Normalization](#)

[2.1. Min-Max Normalization](#)

[2.2. Z-Score Standardization](#)

[2.3. Log Transformation](#)

[2.4. Robust Scaling](#)

[2.5. Max Absolute Scaling](#)

[Summary](#)

[References](#)

Normalization (also known as **feature scaling**) is an important preprocessing step in data analysis and, especially, in machine learning. While humans can easily compare values with different units and scales, most machine learning algorithms rely on mathematical calculations such as distance, gradients, or optimization. As a result, features with much larger values can unintentionally dominate those with smaller values, leading to poor model performance.

To better demonstrate normalization techniques, we will expand our previous student dataset with additional numerical and categorical features. This richer dataset more closely resembles real-world data and will also be used in the next section on **encoding**.

```

import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]

n = 20

df = pd.DataFrame({
    "Name": names,
    "Gender": np.random.choice(["Male", "Female"], n),
    "Age": np.random.randint(18, 21, n),
    "Height": np.random.randint(150, 191, n),
    # cm
    "Weight": np.random.randint(45, 91, n),
    # kg
    "Major": np.random.choice(majors, n),
    "Scholarship": np.random.choice(["Yes", "No"], n),
    "English Level": np.random.randint(1, 6, n),
    # 1-5
    "Study Hours": np.random.randint(5, 31, n),
    # hours/week
    "Attendance": np.random.randint(60, 101, n),
    # %
    "Distance": np.round(np.random.uniform(0.5, 25, n), 1),
    # km
    "Previous Exam": np.random.randint(40, 101, n),
    # score
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

```

```
} )
```

```
print(df.head(10))
```

1. Why Do We Need Normalization?

Imagine we want to build a machine learning model using the following features:

Feature	Typical Range
Age	18 - 20
Height	150 - 190
Weight	45 - 90
Study Hours	5 - 30
Attendance	60 - 100
Distance	0.5 - 25
GPA	2.0 - 4.0

Notice that these features have completely different scales.

For humans, this is not a problem—we naturally understand that **180 cm** and **3.8 GPA** measure different things. However, machine learning algorithms simply treat these values as numbers. Algorithms that depend on distances or gradients (such as **KNN**, **K-Means**, **SVM**, Neural Networks, and Logistic Regression) may give much greater importance to features with larger numerical values.

For example, consider two students:

Student	Height	GPA
A	170	3.8
B	175	3.4

The height differs by **5**, while the GPA differs by only **0.4**. Without normalization, the height difference contributes much more heavily to distance calculations, even though GPA may be equally or more important.

Normalization places numerical features onto a similar scale so that each feature contributes more fairly during analysis or model training.

2. Common Types of Normalization

Before normalizing the data, we first need to identify which columns are numerical. Since normalization only applies to numeric data, Pandas provides the convenient `select_dtypes()` method for selecting columns based on their data types.

- This method is especially useful when working with large datasets containing dozens or even hundreds of features.

```
# Select numerical columns
numerical = df.select_dtypes(include=np.number)
print(numerical.columns)
```

```
Index(['Age', 'Height', 'Weight', 'English Level',
      'Study Hours', 'Attendance', 'Distance',
      'Previous Exam', 'GPA'],
      dtype='object')
```

- Likewise, we can retrieve the categorical columns.

```
categorical = df.select_dtypes(exclude=np.number)
print(categorical.columns)
```

```
Index(['Name', 'Gender', 'Major', 'Scholarship'],
      dtype='object')
```

Unlike some machine learning libraries, **Pandas does not provide built-in normalization functions**. However, because Pandas supports efficient vectorized operations, implementing common normalization techniques only requires a few lines of code. This also gives us a better understanding of what each method is doing behind the scenes.

2.1. Min-Max Normalization

Min-Max normalization rescales each feature into a fixed range, typically **0 to 1**.

```
def min_max_normalize(series):  
    return (series - series.min()) / (series.max() - series.min())  
  
# Normalize every numerical column  
df_minmax = df.copy()  
  
for col in numerical.columns:  
    df_minmax[col] = min_max_normalize(df[col])  
  
print(df_minmax.head())
```

Advantages

- Produces values between **0 and 1**.
- Easy to interpret.
- Commonly used for neural networks.

2.2. Z-Score Standardization

Z-Score normalization transforms each feature so that it has: Mean = 0 and Standard Deviation = 1

```
def z_score_normalize(series):  
    return (series - series.mean()) / series.std()  
  
df_zscore = df.copy()  
  
for col in numerical.columns:  
    df_zscore[col] = z_score_normalize(df[col])  
  
print(df_zscore.head())
```

Advantages

- Handles different feature scales well.

- Frequently used in statistics and machine learning.
- Less affected by varying feature ranges than Min-Max normalization.

2.3. Log Transformation

Log transformation compresses large values and reduces skewness in highly skewed data.

```
def log_transform(series):  
    return np.log1p(series)  
  
df_log = df.copy()  
  
# Only apply to positive-valued columns  
for col in numerical.columns:  
    if (df[col] >= 0).all():  
        df_log[col] = log_transform(df[col])  
  
print(df_log.head())
```

Advantages

- Reduces the influence of extremely large values.
- Makes skewed distributions more symmetric.
- Common for financial and population data.

2.4. Robust Scaling

Robust Scaling uses the **median** and **interquartile range (IQR)** instead of the mean and standard deviation.

```
def robust_scale(series):  
    q1 = series.quantile(0.25)  
    q3 = series.quantile(0.75)  
    iqr = q3 - q1
```

```

        return (series - series.median()) / iqr

df_robust = df.copy()

for col in numerical.columns:
    df_robust[col] = robust_scale(df[col])

print(df_robust.head())

```

Advantages

- Resistant to outliers.
- Works well when the dataset contains extreme values.

2.5. Max Absolute Scaling

Max Absolute Scaling divides every value by its maximum absolute value.

```

def max_absolute_scale(series):
    return series / series.abs().max()

df_maxabs = df.copy()

for col in numerical.columns:
    df_maxabs[col] = max_absolute_scale(df[col])

print(df_maxabs.head())

```

Advantages

- Preserves zero values.
- Simple and computationally efficient.
- Useful for sparse datasets.

Summary

Normalization (or feature scaling) adjusts numerical features so they are on comparable scales. Many machine learning algorithms, particularly those based on distances or optimization, perform better when numerical features are normalized.

- Use `select_dtypes(include=np.number)` to automatically identify all numerical columns before applying normalization.
- Although **Pandas does not include built-in normalization methods**, its vectorized operations make implementing common scaling techniques straightforward.
- The most widely used normalization methods are:
 - **Min-Max Normalization** – rescales values to a fixed range (typically 0–1).
 - **Z-Score Standardization** – centers data around a mean of 0 with a standard deviation of 1.
 - **Log Transformation** – compresses large values and reduces skewness.
 - **Robust Scaling** – uses the median and interquartile range, making it resistant to outliers.
 - **Max Absolute Scaling** – scales values by their maximum absolute value while preserving zeros.
- In practice, **Min-Max Normalization** and **Z-Score Standardization** are the two techniques used most frequently and will be the primary focus throughout this book.

References

<https://towardsdatascience.com/data-normalization-with-pandas-and-scikit-learn-7c1cc6ed6475/>

<https://www.geeksforgeeks.org/pandas/python-scaling-numbers-column-by-column-with-pandas/>

Next Section:

 [6.2. Data Encoding](#)